

Relatório Técnico de Engenharia: Coexistência ESP-NOW, BLE GATT e BT Classic em Arquitetura ESP32-PICO-V3-02

Sumário Executivo

A engenharia de sistemas embarcados focada na convergência de múltiplos protocolos de radiofrequência (RF) frequentemente esbarra em limitações físicas severas inerentes à topologia do silício. Este relatório apresenta uma dissecação exaustiva do ecossistema do wearable de comunicação assistiva **GESTUUM**, o qual opera sobre uma rede de dois dispositivos M5StickC Plus2, baseados no System-in-Package (SiP) ESP32-PICO-V3-02. O desafio central reside na incapacidade do microcontrolador principal (Sensor A) de orquestrar, simultaneamente e sem degradação de sinal, três pilhas de comunicação distintas: Wi-Fi (ESP-NOW no Canal 13), Bluetooth Low Energy (BLE GATT Server) e Bluetooth Classic (A2DP Source).

A análise técnica aprofundada das tentativas de resolução revela que a falha catastrófica — materializada pelo pânico de núcleo `osi_thread_post_event thread.c:431` — não é um erro de lógica de aplicação, mas uma falha arquitetural crônica na camada de interface do sistema operacional (OSI) do stack Bluedroid, manifesta ao se tentar desativar dinamicamente o periférico Bluetooth sob o framework Arduino-ESP32 (baseado no ESP-IDF v4.4.x). Adicionalmente, a saturação do rádio evidenciada na queda de conexão do Progressive Web App (PWA) é um sintoma clássico de estrangulamento da Multiplexação por Divisão de Tempo (TDM).

A recomendação definitiva deste documento aponta para uma reestruturação arquitetural assimétrica: a delegação integral das operações do servidor BLE GATT para o dispositivo periférico (Sensor B), promovendo-o a um *Gateway BLE-to-ESP-NOW*. Esta manobra isola as cargas isócronas do A2DP das rajadas assíncronas do BLE em hardwares separados, restaurando a estabilidade determinística da rede. O documento detalha as metodologias de fragmentação na camada de aplicação (L7) necessárias para rotear cargas de 3842 bytes sobre as restrições nativas do ESP-NOW, as vantagens inquestionáveis da transição para a pilha Apache NimBLE no Sensor B, os ajustes rigorosos no `sdkconfig` via PlatformIO, e um plano estrito de validação empírica.

1. Fundamentos Físicos e Limitações do Rádio no ESP32-PICO-V3-02

A compreensão do comportamento anômalo da rede exige uma análise primária do hardware subjacente. O ESP32-PICO-V3-02, enraizado na tecnologia de 40 nm da TSMC, encapsula o silício ESP32 (ECO V3), 8 MB de memória Flash SPI e 2 MB de PSRAM.¹ Embora o dispositivo suporte o padrão 802.11 b/g/n, Bluetooth Classic (BR/EDR) e Bluetooth Low Energy (BLE), a restrição fundamental da arquitetura é a existência de um único transceptor de RF (Radio Frequency Front-End) e uma única antena integrada.⁴

1.1 A Mecânica da Multiplexação por Divisão de Tempo (TDM)

A operação simultânea de Wi-Fi e Bluetooth em um chip de rádio único é uma abstração de software sustentada por um módulo de coexistência.⁶ Como os protocolos ocupam a mesma banda ISM de 2.4 GHz, o hardware não pode emitir e escutar simultaneamente, nem pode processar modulações de Wi-Fi (OFDM/DSSS) e Bluetooth (GFSK) no mesmo instante de clock.⁷

O módulo de coexistência da Espressif fatiará o tempo (Time-Division Multiplexing - TDM), alocando fatias de milissegundos para cada protocolo de acordo com um sistema de prioridades predefinido.⁶ O desempenho e a estabilidade da rede dependem diretamente do perfil de tráfego injetado no escalonador.

Protocolo	Categoria de Tráfego	Exigência de Rádio	Tolerância a Interrupções (TDM)
BT Classic (A2DP)	Isócrono (Fluxo contínuo)	Alta. Ocupa faixas massivas de tempo para codificação SBC e envio de frames de áudio. ⁵	Baixíssima. A preempção agressiva resulta em <i>stuttering</i> (cortes audíveis). ⁸
BLE (GATT)	Assíncrono / Janelas Rígidas	Baixa a Média. Troca rápida de pacotes durante os intervalos de conexão. ⁹	Baixa. Se o rádio estiver ocupado no momento exato do <i>Connection Event</i> , o pacote cai. ¹⁰
Wi-Fi (ESP-NOW)	Assíncrono (Action Frames)	Muito Baixa (Rajadas ultracurtas de ~2 ms). ⁸	Média. Permite contenção e retentativas MAC, mas satura buffers se bloqueado. ⁸

1.2 O Conflito de Tráfego no "Sensor A" (Tentativa 1)

Na "Tentativa 1" relatada, a coexistência total foi exigida do Sensor A. A física do escalonador colapsou sob a seguinte carga:

1. O Sensor A precisa manter o link A2DP com a caixa de som, consumindo enormes blocos isócronos de transmissão.
2. O Sensor A recebe dados de IMU do Sensor B via ESP-NOW a 50Hz. Isso exige que o rádio comute para o Canal 13 de Wi-Fi e entre em estado de escuta a cada 20 milissegundos.
3. O Sensor A tenta operar como um Servidor GATT BLE. O PWA no Chrome solicita a leitura de `list_trainable` (3842 bytes), o que força o envio de múltiplos pacotes em fila sobre uma conexão de baixa energia.

Quando a transmissão do BLE Notify de 3842 bytes é iniciada, o rádio precisa honrar as estritas janelas de tempo do BLE. No entanto, o fluxo incessante de pacotes ESP-NOW chegando a cada 20 ms força o árbitro de coexistência a fragmentar severamente o tempo do rádio.⁵ A documentação arquitetural da Espressif classifica a performance de "BLE GATT em operação simultânea com fluxo isócrono e tráfego intenso de Wi-Fi" como instável ou não suportada sem perda drástica de *throughput*.⁴ O cliente BLE sofre desconexão porque o rádio ESP32, preso servindo ao A2DP e ao ESP-NOW, perde as janelas âncora de *keep-alive* do Bluetooth Low Energy, violando o tempo limite de supervisão (Supervision Timeout) do protocolo, momento no qual a pilha do dispositivo host (Android/iOS/ChromeOS) encerra a conexão sem aviso prévio.

2. Diagnóstico da Exceção Fatal: O Bug `osi_thread_post_event`

Para contornar o engarrafamento de RF, a "Tentativa 2" introduziu um conceito de transição de estados: desativar o BLE dinamicamente via `BLEDevice::deinit(true)` durante a fase de treinamento, religando o ESP-NOW, para posteriormente religar o BLE. O resultado foi um *Crash* cerca de 1 segundo após o comando de desativação, assinado por `assert failed: osi_thread_post_event thread.c:431 (event!= NULL && event->thread!= NULL)`.

2.1 Dissecação do Comportamento do Core e da Camada OSI

O erro não reside no escopo da aplicação (firmware principal), mas nas profundezas da máquina de estados do Bluedroid, que é a pilha Bluetooth padrão implementada na API do Arduino-ESP32.⁵ A pilha Bluedroid é fortemente multithreaded, apoiando-se na camada OSI (Operating System Interface) para orquestrar mensagens entre os eventos originados pelo Hardware (Controller) e o processamento de Software (Host) usando as primitivas do FreeRTOS.¹³

Quando o comando `BLEDevice::deinit(true)` é invocado na biblioteca do Arduino, ele executa

uma cascata de derrubada do stack ¹⁴:

1. `esp_bluedroid_disable()`
2. `esp_bluedroid_deinit()`
3. `esp_bt_controller_disable()`
4. `esp_bt_controller_deinit()`
5. `esp_bt_controller_mem_release(ESP_BT_MODE_BTDM)`

A interface host-controlador (HCI) do Bluetooth opera de forma altamente assíncrona. Quando o fechamento é solicitado, o cliente BLE (PWA) é desconectado. O rádio emite um pacote de finalização e a camada de hardware agenda um evento de `HCI_DISCONNECT_COMPLETE` a ser processado pela `bt_task` (a thread principal do controlador Bluetooth).¹³

O vício arquitetural na versão de software do ESP-IDF subjacente ao Arduino-ESP32 2.0.17 (`espressif32@6.12.0`) ocorre devido a uma condição de corrida (Race Condition). A chamada de API destrói o manipulador (handler) da thread OSI e limpa as estruturas de controle em memória de maneira quase imediata. Contudo, o hardware físico ou camadas inferiores do L2CAP (Logical Link Control and Adaptation Protocol) continuam postando eventos residuais relativos ao desligamento do link de rádio na fila dessa thread agora inexistente.¹⁵ Ao chamar a função `osi_thread_post_event()`, a validação do ponteiro da thread falha (linha 431 em `thread.c`), pois `event->thread` é nulo.¹³ A macro `assert()` captura a irregularidade e invoca um pânico imediato do núcleo, paralisando a CPU 0.¹⁵

2.2 Inviabilidade de Workarounds no Bluedroid (Arduino-ESP32 2.x)

As investigações confirmam que não existe um *workaround* de código de aplicação robusto para esse comportamento no branch do Bluedroid da versão 2.x do core Arduino.¹⁴

- **Mitigação com Delays:** Injetar atrasos (`vTaskDelay`) antes do *deinit* ou tentar aguardar que todas as *callbacks* de desconexão encerrem não é determinístico, pois a thread de background pode suspender arbitrariamente.
- **O Parâmetro deinit(false):** Invocar a desinicialização passando o argumento `false` evita a rotina `esp_bt_controller_mem_release()`. Sem liberar a memória, as estruturas fundamentais sobrevivem, e o crash na thread cessa.¹⁴ Contudo, a contrapartida é letal para sistemas com ciclo de vida prolongado: **Memory Leak** (Vazamento de Memória). Em fóruns de arquitetura ESP32, há consenso de que o `loop init()` / `deinit(false)` drena cerca de 1 KB a 10 KB de heap por iteração.¹⁸ Se a usuária do wearable entrar e sair repetidamente do modo de treino, o microcontrolador fatalmente sofrerá corrupção de memória ou congelamento sistêmico pela falência dos alocadores (`malloc failure`).

Com a restrição de que o framework deve permanecer em `espressif32@6.12.0` (amarrado à arquitetura IDF v4.4.x), a tentativa de ligar/desligar a pilha Bluedroid dinamicamente sob estresse deve ser formalmente abandonada. A pilha Bluedroid exige ser inicializada no *boot* e mantida ligada até o corte de energia da placa.¹⁹

3. Direcionamento Arquitetural: O Sensor B como Gateway BLE

Se o rádio único do Sensor A não comporta o tráfego heterogêneo (TDM congestionado) e a máquina de estados não pode reativar a antena (Crash do Bluedroid), o paralelismo físico apresenta-se como a única solução sustentável. A hipótese de utilizar o **Sensor B como ponte (Bridge)** é perfeitamente viável, extensivamente fundamentada na literatura técnica de *Internet of Things* (IoT) de baixa potência e representa um paradigma escalável para contornar gargalos de rádio único.⁴

3.1 Benefícios do Isolamento de Domínios de Colisão

Ao migrar o servidor BLE GATT para o dispositivo da mão secundária (Sensor B), o sistema segrega as naturezas de tráfego de maneira harmoniosa com os mecanismos de rádio²²:

- **Domínio do Sensor A:** Passa a abrigar apenas Wi-Fi (ESP-NOW) e Bluetooth Classic (A2DP). Esta é uma configuração de coexistência documentada e estável. Sem as interrupções de conexão rígidas do BLE, o escalonador interno equilibra tranquilamente o envio contínuo das amostras SBC do áudio com as recepções intermitentes do ESP-NOW vindas do Sensor B.²²
- **Domínio do Sensor B:** Passa a abrigar Wi-Fi (ESP-NOW) em modo STA e Bluetooth Low Energy (BLE GATT). A ausência absoluta do protocolo A2DP neste nó remove a demanda por fatias maciças e contínuas de tempo de rádio. O tráfego BLE interage de modo altamente eficiente com o protocolo ESP-NOW, visto que ambos admitem interrupções diminutas e recuperações ágeis de link.⁶ Projetos como o *OpenMQTTGateway* e pontes seriais *ESPNow-to-BLE* comprovam a estabilidade térmica e operacional deste pareamento em módulos do porte do ESP32-PICO.²¹

3.2 Topologia de Salto (Hop Topology) e Análise de Latência

O fluxo de dados da arquitetura refatorada estabelece que o PWA dialogará exclusivamente com o Sensor B via Web Bluetooth. As requisições de controle e treino (`train_start`, `get_config`) e o tráfego resultante sofrerão um "salto" de rádio:

↔ (BLE GATT) ↔ ↔ (ESP-NOW Canal 13) ↔

A introdução do Sensor B como interceptador adiciona uma latência de processamento perfeitamente tolerável para interação humano-computador. Estudos práticos documentam que a transmissão unidirecional baseada em pacotes da ordem de 16 a 250 bytes no ESP-NOW oscila entre **1.5 ms e 3.0 ms**.⁸ Mesmo ao se considerar processamento e decodificação na camada de aplicação, o atraso injetado pelo *bridge* rondará os 5 a 10 milissegundos. Considerando que os tempos de resposta típicos requeridos pela especificação Bluetooth Low Energy para a camada de aplicativo dependem do intervalo de conexão (normalmente de 30 ms a 45 ms) imposto pelo dispositivo mestre (Smartphone/PC executando o PWA)²⁶, o gargalo

da latência final permanecerá no link Bluetooth, tornando o salto do ESP-NOW transparente ao utilizador final.

3.3 A Arquitetura de Paginação para Transmissões Gigantes (3842 Bytes)

A viabilização do Gateway no Sensor B invoca um entrave matemático agudo no protocolo subjacente de Wi-Fi: o transporte da matriz de dados `list_trainable`, pesando 3842 bytes.

Em conexões BLE nativas, a Unidade Máxima de Transmissão (MTU) negocia dinamicamente e a camada L2CAP subjacente fragmenta automaticamente *payloads* enormes sobre múltiplos pacotes de rádio.⁵ No entanto, o ESP-NOW versão 1.0 (a implementação atrelada ao IDF 4.4.x em uso via framework Arduino-ESP32 2.0.17) detém um limite rígido inserido na especificação MAC de **250 bytes úteis por pacote** em *Action Frames*.¹¹ A submissão direta do vetor de 3842 bytes à rotina `esp_now_send()` resultará inexoravelmente na truncagem silente dos dados a 250 bytes ou no lançamento contínuo de uma exceção do tipo `Packet too large` no subsistema de transporte.¹¹

A resolução técnica obriga a implementação de uma fragmentação reversa na Camada de Aplicação (L7).

3.3.1 Algoritmo de Fragmentação (Chunking)

O Sensor A deve fatiar o *payload* antes de acionar a antena, e o Sensor B deve recombinar o vetor em seu *heap* para posterior submissão atômica à pilha BLE. A métrica estrita delineia que a carga (chunk payload) deve ser limitada a um teto seguro (ex.: 230 a 240 bytes) de modo a ceder margem para o cabeçalho de metadados de envelopamento.

Parâmetro de Fragmentação	Valor Lógico
Tamanho do Objeto a Transmitir	3842 Bytes
Teto Seguro de Carga (Payload)	240 Bytes por pacote ¹¹
Total de Fragmentos (Chunks)	17 Pacotes (16 x 240 Bytes + 1 x 2 Bytes)

A definição estrutural orientada a objetos (struct) serializada demanda os atributos delineadores:

C++

```
typedef struct __attribute__((packed)) {
    uint8_t packet_type;    // Ex: 0x02 para Transferência em Massa
    uint16_t total_payload; // Comprimento global (3842)
    uint8_t total_chunks;   // Agregado de partições (17)
    uint8_t current_chunk;   // Índice de rastreio seqüencial (0 a 16)
    uint16_t chunk_length;   // Bytes transportados nesta fatia
    uint8_t payload;         // Matriz de transporte de dados
} espnow_large_transfer_t;
```

3.3.2 Controle de Fluxo Rígido (Acknowledge)

É imperativo que a máquina de estados não sature a *buffer* de transmissão TCP/IP do LwIP ou as filas anelares do driver de Wi-Fi submetendo os 17 *chunks* através de um laço síncrono for sequencial.²⁹ Emissores em alta velocidade levam à contenção do espectro e fragmentação distorcida.¹¹

A documentação da Espressif delibera enfaticamente a metodologia: *"Caso exista uma grande massa de dados ESP-NOW a transacionar, deve-se invocar esp_now_send() e aguardar a consumação do disparo; o intervalo curto ou a omissão desta espera conduz ao transtorno do callback de envio"*.¹¹ O protocolo deve funcionar em cascata (*Stop-and-Wait ARQ* modificado): o Sensor A injeta o Chunk 0, cessa a atividade e retorna ao escopo do *loop*. A interrupção assíncrona da placa engatilhará a rotina OnDataSent repassando a bandeira ESP_NOW_SEND_SUCCESS. Somente sob a aferição positiva o *dispatcher* liberará o Chunk 1.

3.3.3 Remontagem no Gateway BLE (Sensor B)

Do lado do receptor, o Sensor B agirá como um alocador transitório. No recebimento do Chunk 0, um bloco dinâmico em memória de 3842 bytes é reservado. À medida que os *callbacks* OnDataRecv se concretizam, os *payloads* são decalcados (memcpy) com o desvio base correspondente a $\text{current_chunk} * 240$. Uma vez recepcionado o último índice (Chunk 16), o Sensor B dispara a notificação maciça à pilha BLE.⁵ Essa abordagem isola inteiramente as complexidades do Web Bluetooth das restrições MTU do rádio local.

4. Avaliação Crítica das Hipóteses e Otimizações de Protocolo

A investigação contemplou questionamentos analíticos a respeito de mecanismos intrínsecos de rádio como as taxas de *Long Range*, priorizações dinâmicas no tráfego, e a conversão de *stack* via bibliotecas de terceiros.

4.1 Rechaço Categórico ao ESP-NOW LR Mode (Long Range)

A ativação da diretiva WIFI_PROTOCOL_LR altera o estado de modulação da interface de Wi-Fi na camada física, abdicando de altas velocidades (como os 1 Mbps nominais da

codificação DSS do 802.11b) em favor de modulações DSSS de espectro muito espalhado para mitigar a atenuação de sinal no ambiente aberto, derrubando a taxa de transmissão bruta para limites ínfimos entre 250 kbps a 500 kbps.³²

No domínio de redes coexistentes, **menor vazão sistêmica implica diretamente num rádio engajado por mais tempo (Airtime Occupation).**³⁴ Transmitir os pacotes (que demorariam ~2 ms a 1 Mbps) exigirá, em LR Mode, o triplo ou quádruplo da janela temporal, o que sequestrará fatalmente os *time slices* que o TDM da Espressif tenta preservar desesperadamente para as atualizações do GATT do Bluetooth. Portanto, o Modo LR causará latência destrutiva, colidirá com as janelas operacionais do BLE e corromperá a integridade do protocolo.

4.2 Impacto Positivo da Sub-Amostragem Temporária do IMU (20Hz)

Uma cadência operativa de 50Hz dita um acionamento do chip rádio a cada 20 milissegundos por parte do Sensor B, unicamente para propagar dados de telemetria.⁷ No cenário do Gateway, embora a latência do ESP-NOW não perturbe o BLE diretamente, a redução condicional desta taxa em momentos vitais revela-se uma tática providencial para o Sensor A (detentor do A2DP).

Caso o firmware intercepte que o *modo de treinamento* foi assinalado e as transmissões pesadas bidirecionais começarão a vigorar, emitir um pacote sinalizador para o Sensor B reduzir a amostragem inercial de 50Hz para **20Hz** (a cada 50 milissegundos) instaura instantaneamente extensos vazios lógicos (*Idle Slices*) na faixa espectral.⁸ Essas "férias" do rádio proveem hiatos mais flexíveis para o algoritmo de TDM encaixar interrupções dos saltos de controle do perfil A2DP e a troca das fragmentações dos 3842 bytes, desanuviando a carga térmica e digital de banda sem incorrer em interrupções forçadas do ecossistema e do controle do alto-falante acoplado à mão dominante.

4.3 A Superioridade Incontestada da Pilha NimBLE (Sensor B)

A transição arquitetural de alocação de servidores GATT do Sensor A para o B abre um precedente vital de aprimoramento de engenharia: o descarte integral da pesada pilha Bluedroid (que permanecerá intocada no Sensor A apenas por abrigar as diretrizes proprietárias A2DP que outras *stacks* não suportam) no Sensor B.¹²

Recomenda-se estritamente o uso da biblioteca open-source NimBLE-Arduino (desenvolvida sobre o host Apache MyNewt NimBLE) em substituição à biblioteca padrão BLEDevice da Espressif.³⁷ As métricas apontam abismos de eficiência e estabilidade que fundamentam essa indicação:

1. **Imunidade ao Memory Leak e Cleanups Críticos:** Diferente do *Spaghetti Code* que envolve o despachante de eventos da camada OSI do Bluedroid, que invariavelmente afunda com o erro `osi_thread_post_event`¹³, a topologia do host NimBLE segrega as suas lógicas de forma cristalina. O desfazimento da pilha BLE e *Advertising* (`NimBLEDevice::deinit()`) limpa as filas sem esgotar as sentinelas de alocação de FreeRTOS nem disparar asserções no núcleo, operando de modo infinitamente mais

- complacente em ambientes onde o Wi-Fi também modula interrupções de prioridade.³⁹
2. **Pegada de Memória (Heap e Flash):** O módulo Bluetooth do PICO-V3-02 não pode despejar livremente memória no PSRAM sob o risco severo de sofrer lentidão aguda de cache (Bug mfix-esp32-psram-cache-issue).¹⁰ Operar exclusivamente na SRAM (DRAM) interna é o arranjo seguro. O stack Bluedroid sequestra impiedosamente até 130 KB da RAM limpa em sua mera inicialização. O NimBLE opera eficientemente ocupando cerca de 50% menos memória, deixando uma pista imensa de RAM intocada para o manipulador de fragmentos L7 dos vetores de 3842 bytes e *arrays* de IMU.³⁷
 3. **Transparência ao Front-end PWA:** A migração de Bluedroid para NimBLE ao nível do C++ exige adaptações semânticas menores no firmware (BLEDevice vira NimBLEDevice, BLECharacteristic vira NimBLECharacteristic, etc.), mas a camada MAC aérea permanece intacta.³⁷ Portanto, a refatoração **não afeta nenhuma linha de código do `app/gestuum-app.html`**. O cliente Web Bluetooth interpretará os GUIDs das características e assinaturas do GATT como se nenhuma modificação tivesse existido na outra extremidade.
 4. **Rapidez de Associação e Throughput Assíncrono:** Ensaios independentes aferiram que a conexão a partir do escaneamento consome impressionantes 623 ms no NimBLE contra estagnados 946 ms do Bluedroid⁴³, denotando rapidez estrita, minimizando a *exposure time* do rádio, o que agrada imensamente as priorizações de contenção do módulo de coexistência em software nativo do IDF e maximiza as taxas brutas de transferências (Throughput) para notificações via Bluetooth.⁴³

5. Engenharia de Configuração: Parâmetros Vitais (`sdkconfig` e `platformio.ini`)

Existe um entrave crônico sobre as intenções de injetar flags de controle na raiz de um projeto que consome o core de terceiros pelo PlatformIO. Alterações passadas por `-D CONFIG_XXX=1` na chave `build_flags` dentro do arquivo `platformio.ini` sob a diretiva fundamental `framework = arduino` são majoritariamente estéreis.⁴⁵ O núcleo do Arduino para o ESP32 provê arquivos bibliotecários de código objeto pré-compilados (.a), e, portanto, restrições e opções arquiteturais fundamentais do sistema operacional base da Espressif — os quais regulamentam as prioridades do hardware TDM e a coexistência entre Wi-Fi e MAC Bluetooth — foram fixadas no momento em que a própria Espressif compilou esses módulos.⁴⁷

Para destravar a habilidade de acessar a matriz fundamental de comportamento de rádio e as prerrogativas térmicas, a configuração do Sensor A e do Sensor B necessitam adotar a integração profunda do IDF sobre o framework de camadas cruzadas:

Ini, TOML

```

; Exemplo adaptado para a raiz do platformio.ini (Sensor A/B)
[env:m5stickc_plus2]
platform = espressif32@6.12.0
board = m5stick-c
framework = arduino, espidf
build_flags =
    -D SENSOR_A
    -D BOARD_HAS_PSRAM
    -mfix-esp32-psram-cache-issue

```

A inserção da tupla *arduino, espidf* orienta o motor CMake local do PlatformIO a compilar toda a imensa árvore do ESP-IDF a partir do zero no computador do usuário, fundindo o núcleo de bibliotecas como componentes subordinados.⁴⁹

A partir deste estágio, executa-se o comando `pio run -t menuconfig` gerando o arquivo `sdkconfig`. Os seguintes perfis internos são taxativos para a orquestração imperturbável da rede conjunta:

Chave do Módulo (Kconfig)	Definição Requerida	Justificativa de Engenharia da Operação
CONFIG_ESP_COEX_SW_COEXIST_ENABLE	=y	Módulo primordial. Ativa o software controlador de tempo do RF e a política de fatiamento condicional (TDM). É absolutamente impensável rodar Wi-Fi e Bluetooth simultaneamente sem esta flag ativada na compilação do rádio. ¹⁰
CONFIG_ESP_COEX_PREFERENCE	BALANCE	A Espressif cede configurações como "WIFI" ou "BT". Na coexistência estressada pelo A2DP com áudio (Sensor A), setar em balanço mitiga travamentos do ESP-NOW enquanto acomoda as janelas rígidas da música. ⁵²

CONFIG_BTDM_CTRL_PINNED_TO_CORE	=0 (Core 0)	Ancoragem de <i>Threads</i> (Pinning). Empurrar estritamente toda a controladora PHY do Bluetooth, os barramentos estáticos HCI e os eventos L2CAP nativos para o Processador Protocolar (PRO CPU / Core 0). ⁶
CONFIG_ESP_WIFI_TASK_CORE_ID	=1 (Core 1)	Atrai os manipuladores LwIP, o decodificador Wi-Fi e as rotinas originadas pelo ESP-NOW de interrupção MAC para o Processador de Aplicação (APP CPU / Core 1). A disparidade de núcleos provê uma barreira contra contenção de processos RTOS de rádio, prevenindo quedas simultâneas de pilha. ⁶
CONFIG_BT_ALLOCATION_FROM_SPIRAM_FIRST	=n (Falso)	Preservação Operacional do ESP32-PICO-V3-02. O chip provê 2MB PSRAM, no entanto, é suscetível a bugs de acesso direto associados a travas inter-núcleos (cache lock). A restrição das matrizes de Bluetooth alocando-se primeiro no chip PSRAM corromperia fluxos rápidos isócronos de áudio. Todas as manipulações do BT devem ocupar firmemente a DRAM. ¹⁰
CONFIG_BT_NIMBLE_ENABLED (Apenas Sensor B)	=y	Desativa em cascata todo o suporte do Bluedroid, alijando o componente e

		promovendo a estrutura modular para atracação do NimBLE OS, garantindo limpeza na Camada de Rádio. ³⁸
--	--	--

6. Plano Sistêmico de Validação de Hipóteses e Riscos

Mutações drásticas na superestrutura de redes demandam um cinturão de proteção composto por testes isolados e mensuráveis antes da imersão nas milhares de linhas do main.cpp (FSM principal). É sugerida a criação de um repositório satélite (tests/poc_gateway/) sem acoplamento à máquina de gestos para aferir o espectro eletromagnético e estabilidade do núcleo, com foco primário em validar as interações Wi-Fi + BLE sem BT Classic na mesma antena e as propriedades dos pacotes segmentados.

6.1 Teste Isolado A: Throughput Bidirecional no Gateway NimBLE

- **Procedimento (Mock):** Crie um script minimalista operando unicamente no Sensor B. Inicialize Wi-Fi em modo WIFI_STA, e levante as camadas ESP-NOW e servidor NimBLE. No servidor BLE, configure uma Característica notificável fictícia e escreva uma matriz estática em blocos contínuos. Em um outro dispositivo (que representaria o A), configure o ESP-NOW para inundar (flood) o Sensor B à taxa pretendida de 50Hz usando as funções esp_now_send com a respectiva assinatura estrutural (16-32 bytes).
- **Métrica de Aceitação:** Via Chrome PWA (Web Bluetooth API), assine as Notificações. Monitore a serial do dispositivo injetor de ESP-NOW e ateste o registro do callback ESP_NOW_SEND_SUCCESS. O sistema não deve demonstrar travamentos (Stall/WDT Reboot) ao longo de testes de rajadas de 10 minutos (stress contínuo), ratificando a harmonia da coexistência na antena unificada.⁵

6.2 Teste Isolado B: Confiabilidade da Engrenagem L7 de Fragmentação de 3842 Bytes

- **Procedimento (Mock):** Sem conexão BLE envolvida, estipule a comunicação puramente ESP-NOW entre os M5StickC Plus2. Dispare um fluxo de comando de controle (train_start) seguido pelo disparo de 17 chunks contendo a sequência pseudoaleatória encapsulada pela struct de pacote apresentada, sob a dinâmica de FSM orientada por Callbacks.
- **Métrica de Aceitação:** Validação da integridade temporal e reconstrução exata da memória. Os blocos remontados no heap destino (malloc) devem igualar-se em hash CRC32 ao injetado pela origem no remetente, atestando resiliência irrefutável aos gargalos do TDM, limitantes de payload do Action Frame 802.11 na variante de biblioteca

1.0 (250 bytes) e eventuais pacotes corrompidos do ambiente de rádio RF local.¹¹

6.3 Teste Isolado C: Análise Profilática do Perfil Térmico e Dreno de Corrente (Bateria)

- **Procedimento:** A carga operacional imposta sobre o diminuto Sensor B (agora orquestrando o MAC Address de Broadcast/Unicast do ESP-NOW junto às emissões de pacotes de anúncio Bluetooth (Advertising Packets) em janelas frequentes) induz picos de correntes abruptos no amplificador linear (LNA/PA) da placa.⁵⁷
- **Métrica de Aceitação:** Avaliação do consumo energético com analisadores lógicos de drenagem ou osciloscópios no pino V-BAT sob bateria de 200 mAh.³⁴ A arquitetura deve sustentar, confortavelmente e em condições estáveis, a dissipação de carga pelas longas jornadas de utilização pretendidas pelo usuário infantil ou docente em sala sem incorrer em alarmes prematuros de subtensão.

6.4 Risco Residual da Manobra Arquitetural

Mesmo com o rigor da transição analítica, restam margens subjacentes passíveis de detecção na operação diária:

1. O uso da FSM condicional baseada na submissão progressiva de blocos para entrega dos 3842 bytes via callback incorre em um processo probabilístico de aumento da latência linear na devolução (A -> B). Se um dos fragmentos enfrentar colisões espúrias (uma vez que os canais ISM em ambientes ruidosos costumam exibir interferência maciça), o *retry* da submissão retarda a entrega integral da carga, provocando potenciais oscilações ligeiras na experiência visual da PWA até o último indicador progressivo estar em sua completude.¹¹
2. Desconexões indesejadas que afetam exclusivamente o emparelhamento central Sensor B e Sensor A devido a oscilações em distâncias marginais poderão incorrer na quebra temporária do elo ESP-NOW, desautorizando silenciosamente atualizações da máquina NimBLE em direção à tela Web por frações de segundo imperceptíveis ao despachador BLE subjacente até uma re-restauração.

7. Referências e Exemplos Fundamentais (Bibliográficas em Plataformas Open-Source)

A aplicabilidade teórica da refatoração em instâncias comutadas e contornando deficiências intrínsecas de coexistência repousa sobre bases sólidas aferidas nos seguintes projetos correlatos de IoT open-source:

- **ESP-NOW-Gateway (aZholtikov):** Destaca-se como *proxy* que compila extensivamente uma malha entre o Wi-Fi inter-Mcu nativo do ESP-NOW para o protocolo MQTT do ecossistema inteligente, provendo código exemplar para as chamadas da fila MAC e resiliência em falhas de pacote intermitente sobre múltiplos nós comutados na linguagem

nativa do core ESP32.⁵⁹

- **OpenMQTTGateway:** O baluarte fundamental da transição RF. O software converte massas de escuta Bluetooth Low Energy espúrias, roteando os fluxos para bases Wi-Fi sem colidir, documentando através da sua extensa árvore baseada na implementação da biblioteca Apache as vantagens esmagadoras do encapsulamento NimBLE em detrimento dos esgotamentos alocativos das origens Bluedroid no manuseio massivo do IoT de baixa latência e consumo em sistemas SoC.²⁴
- **esp32-ble-gateway (kind3r):** Um *proxy* GATT sobre o ESP32 operante via *sockets*, que serve como base formadora na elucidação dos arranjos dos domínios da preempção de RF, detalhando arquiteturas limpas com AES que superam a fadiga subjacente na junção entre subsistemas simultâneos na CPU.²⁰
- **Discussão Técnica sobre o Paradigma de Coexistência (pschatzmann/ESP32-A2DP):** Evidencia categoricamente, pela análise analítica contínua da comunidade nas *issues* do repositório, que embora o tráfego da API BTDM do BLE funcione pacificamente em pareamento (e em balanço de prioridades modulares de TDM), o esgotamento do tráfego proveniente da música (BT Classic) estralça os *slots* remanescentes e instabiliza o tempo exigido para pacotes assíncronos extras da rede 802.11 da Espressif, referendando a tese de separação imposta.⁸

8. Conclusões Finais

O cerne da degradação dos atributos sem fio e os travamentos do ecossistema GESTUUM são patologias inevitáveis do silício sob a exigência imposta pelo firmware. O subsistema TDM da Camada Coexistente do ESP32-PICO-V3-02 entra em colapso devido a impossibilidade eletromagnética de manter transmissões pesadas e constantes isócronas no protocolo A2DP e escutar modulações simultâneas em 50Hz originadas pela interrupção ESP-NOW, enquanto concorre com prazos irredutíveis e rígidos do fatiamento GATT oriundas do Bluetooth Low Energy para a entrega massiva e paralela de quase 4 KB de tráfego de dados fragmentados sem induzir cortes de áudio ou desconexões.⁴

A desestabilização da Camada Intermediária do Sistema Operacional (OSI) documentada por exaustivas asserções de memória de contexto `osi_thread_post_event` não pode ser re-sintetizada mediante simples artifícios e transições estáticas, dadas as dependências herdadas na *lib* do framework Arduino (ESP-IDF 4.x), impondo a inviabilidade da desativação em ciclo contínuo sem comprometer a estrutura total da Heap com severos Memory Leaks iterativos (cujas resoluções só tangenciam o horizonte nativo e instável das migrações do núcleo 3.x em adiante).¹³

Com amparo da totalidade da literatura analisada, **a recomendação definitiva orienta-se rumo ao acoplamento do dispositivo periférico (Sensor B) como Gateway de tradução (BLE ↔ ESP-NOW)**. A manobra remove substancialmente a carga do rádio da mão dominante, delegando a responsabilidade passiva de comunicações web e fragmentações em L7 à malha externa e assegurando as prerrogativas térmicas e o contínuo determinismo dos protocolos

acoplados no hardware de tempo de latência não percebido pelo utilizador. Adote as configurações recomendadas relativas à ancoragem restrita do núcleo (CONFIG_BTDM_CTRL_PINNED_TO_CORE_0), e migre para a infraestrutura Apache NimBLE (NimBLE-Arduino) no ambiente escravo de modo a selar o fluxo de dados em excelência e estabilidade imutável, garantindo de fato o viés da portabilidade assíncrona educacional da aplicação em sala de aula do wearable PWA.

Referências citadas

1. ESP32-PICO Series - Espressif Documentation, acessado em maio 1, 2026, https://documentation.espressif.com/esp32-pico_series_datasheet_en.pdf
2. ESP32PICOV302 - Datasheet - Adafruit, acessado em maio 1, 2026, https://cdn-learn.adafruit.com/assets/assets/000/117/489/original/esp32-pico-v3-02_datasheet_en.pdf?1673379076
3. ESP32 Pico V3 02 Datasheet - Adafruit, acessado em maio 1, 2026, <https://cdn-shop.adafruit.com/product-files/5395/ESP32-PICO-V3-02-Espressif-Systems-datasheet-147573222.pdf>
4. Noob question: how can I determine if a module can support simultaneous WiFi and BLE : r/Esphome - Reddit, acessado em maio 1, 2026, https://www.reddit.com/r/Esphome/comments/1hahawc/noob_question_how_can_i_determine_if_a_module_can/
5. ESP32 Bluetooth Classic vs BLE: When to Use Which Protocol - Zbotic, acessado em maio 1, 2026, <https://zbotic.in/esp32-bluetooth-classic-vs-ble-when-to-use-which-protocol/>
6. RF Coexistence - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation, acessado em maio 1, 2026, <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/coexist.html>
7. How does BLE/WiFi-Coexistence in the ESP32 work?, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=1464>
8. a2dp sink and WiFi coexistence - ESP32 Forum, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=6511>
9. Bluetooth Classic & BLE with ESP32 - DroneBot Workshop, acessado em maio 1, 2026, <https://dronebotworkshop.com/esp32-bluetooth/>
10. coexistence - - — ESP-FAQ latest documentation, acessado em maio 1, 2026, <https://docs.espressif.com/projects/esp-faq/en/latest/software-framework/coexistence.html>
11. ESP-NOW - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation - Espressif Systems, acessado em maio 1, 2026, https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
12. Introduction - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation, acessado em maio 1, 2026, <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/ble/overview.html>
13. [TW#26982] Bluetooth LE: Core 0 panic'ed on bt_controller_disable w. client

- connected OR esp_ble_set_encryption called · Issue #2626 · espressif/esp-idf - GitHub, acessado em maio 1, 2026, <https://github.com/espressif/esp-idf/issues/2626>
14. ESP32 BLE not turning off - Stack Overflow, acessado em maio 1, 2026, <https://stackoverflow.com/questions/78932872/esp32-ble-not-turning-off>
 15. esp_bluedroid_deinit() crashes [SOLVED] - ESP32 Forum, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=18596>
 16. How to reinitialize BLE? - ESP32 Forum, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=5426>
 17. "BLEDevice::deinit(false);" does NOT work properly (on v3.3.1) #11906 - GitHub, acessado em maio 1, 2026, <https://github.com/espressif/arduino-esp32/issues/11906>
 18. Deinit function in ble - Programming - Arduino Forum, acessado em maio 1, 2026, <https://forum.arduino.cc/t/deinit-function-in-ble/946200>
 19. BLEScan crashes whole ESP32 on second call, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=14413>
 20. kind3r/esp32-ble-gateway - GitHub, acessado em maio 1, 2026, <https://github.com/kind3r/esp32-ble-gateway>
 21. ESPNow to Homeassistant/MQTT Gateway : 4 Steps - Instructables, acessado em maio 1, 2026, <https://www.instructables.com/ESPNow-to-HomeassistantMQTT-Gateway/>
 22. Wifi/BT co-existence mode for A2DP streaming · Issue #218 · espressif/esp-hosted - GitHub, acessado em maio 1, 2026, <https://github.com/espressif/esp-hosted/issues/218>
 23. ESP-NOW Wireless Communication Protocol - Espressif Systems, acessado em maio 1, 2026, <https://www.espressif.com/en/solutions/low-power-solutions/esp-now>
 24. Bluetooth ESP32 gateway | Theengs OpenMQTTGateway v1.8.1, acessado em maio 1, 2026, <https://docs.openmqttgateway.com/use/ble.html>
 25. Looking for solution to connect multiple esp with low latency as reaction timer. : r/esp32, acessado em maio 1, 2026, https://www.reddit.com/r/esp32/comments/10a59sy/looking_for_solution_to_connect_multiple_esp_with/
 26. NimBLE causes crash on ESP32 - Programming - Arduino Forum, acessado em maio 1, 2026, <https://forum.arduino.cc/t/nimble-causes-crash-on-esp32/1038736>
 27. espnow — support for the ESP-NOW wireless protocol — MicroPython 1.22 documentation, acessado em maio 1, 2026, <https://docs.openmv.io/library/espnow.html>
 28. espnow.packet_transport: exceed packet size error when chunking data #13151 - GitHub, acessado em maio 1, 2026, <https://github.com/esphome/esphome/issues/13151>
 29. transferring large chunks of data in websocket : r/esp32 - Reddit, acessado em maio 1, 2026, https://www.reddit.com/r/esp32/comments/1ktma4a/transferring_large_chunks_of_data_in_websocket/

30. Packet fragmentation during large writes · Issue #152 · esp-rs/esp-wifi-sys - GitHub, acessado em maio 1, 2026, <https://github.com/esp-rs/esp-wifi-sys/issues/152>
31. esp-idf/examples/bluetooth/bluedroid/ble/ble_throughput/throughput_server/README.md at master - GitHub, acessado em maio 1, 2026, https://github.com/espressif/esp-idf/blob/master/examples/bluetooth/bluedroid/ble/ble_throughput/throughput_server/README.md
32. Exploring wireless communication protocols on ESP32 platform for outdoor applications, acessado em maio 1, 2026, <https://developer.espressif.com/blog/esp-now-for-outdoor-applications/>
33. Can someone with ESP Now LR mode experience verify this code? : r/esp32 - Reddit, acessado em maio 1, 2026, https://www.reddit.com/r/esp32/comments/1q6m2gw/can_someone_with_esp_now_lr_mode_experience/
34. A comparison of wireless protocols for battery powered embedded devices, acessado em maio 1, 2026, <https://www.marending.dev/notes/esp-protocol/>
35. Comparative Performance Study of ESP-NOW, Wi-Fi, Bluetooth Protocols based on Range, Transmission Speed, Latency, Energy Usage and Barrier Resistance - ResearchGate, acessado em maio 1, 2026, https://www.researchgate.net/publication/355656368_Comparative_Performance_Study_of_ESP-NOW_Wi-Fi_Bluetooth_Protocols_based_on_Range_Transmission_Speed_Latency_Energy_Usage_and_Barrier_Resistance
36. ESP Host Major Feature Support Status - ESP32 - Espressif Systems, acessado em maio 1, 2026, <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/ble/host-feature-support-status.html>
37. ESP32 Bluetooth Low Energy Tutorial: BLE GATT Server - Zbotic, acessado em maio 1, 2026, <https://zbotic.in/esp32-bluetooth-low-energy-tutorial-ble-gatt-server/>
38. NimBLE-based Host APIs - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation, acessado em maio 1, 2026, <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/bluetooth/nimble/index.html>
39. h2zero/esp-nimble-cpp • v2.1.0 - ESP Component Registry - Espressif Systems, acessado em maio 1, 2026, <https://components.espressif.com/components/h2zero/esp-nimble-cpp/versions/2.1.0/changelog?language=>
40. ESP32 Platform - ESPHome - Smart Home Made Simple, acessado em maio 1, 2026, <https://esphome.io/components/esp32/>
41. Can BLE be made to use PSRAM? · Issue #3506 · espressif/arduino-esp32 - GitHub, acessado em maio 1, 2026, <https://github.com/espressif/arduino-esp32/issues/3506>
42. advantages of bludroid and nimble over each other - ESP32 Forum, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=16094>
43. Bluedroid vs NimBLE, and BleKeyboard | Details | Hackaday.io, acessado em maio 1, 2026,

- <https://hackaday.io/project/177896/log/241437-bluedroid-vs-nimble-and-blekeyboa rd>
44. Unexpected NimBLE GATT performance compared to Bluedroid (IDFGH-11677) · Issue #12789 · espressif/esp-idf - GitHub, acessado em maio 1, 2026, <https://github.com/espressif/esp-idf/issues/12789>
 45. Using Arduino Framework Point to Custom sdkconfig.h file - PlatformIO Community, acessado em maio 1, 2026, <https://community.platformio.org/t/using-arduino-framework-point-to-custom-sdkconfig-h-file/12622>
 46. Modifying Arduino ESP32 setup with menuconfig - espressif32 - PlatformIO Community, acessado em maio 1, 2026, <https://community.platformio.org/t/modifying-arduino-esp32-setup-with-menuconfig/20040>
 47. Problem adding CONFIG_ADC to build flags in platformio.ini, acessado em maio 1, 2026, <https://community.platformio.org/t/problem-adding-config-adc-to-build-flags-in-platformio-ini/36805>
 48. Right way for custom SDKCONFIG per environment - PlatformIO Community, acessado em maio 1, 2026, <https://community.platformio.org/t/right-way-for-custom-sdkconfig-per-environment/27546>
 49. Make tweaks in sdkconfig also possible in Arduino · Issue #9456 · espressif/arduino-esp32, acessado em maio 1, 2026, <https://github.com/espressif/arduino-esp32/issues/9456>
 50. Arduino as ESPIF Component in PlatformIO - solution that works, acessado em maio 1, 2026, <https://community.platformio.org/t/arduino-as-espif-component-in-platformio-solution-that-works/36542>
 51. I want to know about the coexistence use of WIFI/BLE on the ESP32 - Arduino Forum, acessado em maio 1, 2026, <https://forum.arduino.cc/t/i-want-to-know-about-the-coexistence-use-of-wifi-ble-on-the-esp32/1155738>
 52. WIFI/BLE Simultaneously - Page 7 - ESP32 Forum, acessado em maio 1, 2026, <https://esp32.com/viewtopic.php?t=6707&start=60>
 53. NimBLE support for ESP32-C6, H2 and future MCU's and why is Bluedroid still the default? #9836 - GitHub, acessado em maio 1, 2026, <https://github.com/espressif/arduino-esp32/discussions/9836>
 54. How much can core 0 handle? : r/esp32 - Reddit, acessado em maio 1, 2026, https://www.reddit.com/r/esp32/comments/vriuwy/how_much_can_core_0_handle/
 55. Esp32 Errata en | PDF | Cpu Cache | Central Processing Unit - Scribd, acessado em maio 1, 2026, <https://www.scribd.com/document/689671262/Esp32-Errata-En>
 56. LFMM: Using BLE and WiFi Together | Production ESP32, acessado em maio 1, 2026, <https://productionesp32.com/posts/lfmm2-ble-wifi-together/>
 57. Is ESP-NOW or BLE more power efficient? : r/esp32 - Reddit, acessado em maio 1, 2026,

https://www.reddit.com/r/esp32/comments/j2j1df/is_espnow_or_ble_more_power_efficient/

58. Preliminary queries on using ESP NOW - Arduino Forum, acessado em maio 1, 2026, <https://forum.arduino.cc/t/preliminary-queries-on-using-esp-now/1287476>
59. Gateway for data exchange between ESP-NOW devices and MQTT broker for ESP8266/ESP32. - GitHub, acessado em maio 1, 2026, <https://github.com/aZholtikov/ESP-NOW-Gateway>
60. WiFi coexistence · Issue #19 · pschatzmann/ESP32-A2DP - GitHub, acessado em maio 1, 2026, <https://github.com/pschatzmann/ESP32-A2DP/issues/19>
61. WIFI and A2DP Coexistence · pschatzmann/ESP32-A2DP Wiki · GitHub, acessado em maio 1, 2026, <https://github.com/pschatzmann/ESP32-A2DP/wiki/WIFI-and-A2DP-Coexistence>